When Successful Runs Do Not Mean Valid Learning: Auditing Silent Reward Contracts in Agentic Reinforcement Learning

Anonymous authors
Paper under double-blind review

Abstract

Reliable agentic reinforcement learning (RL) depends on more than a job exiting successfully. Its pipelines connect multi-turn traces, tool results, evaluators, rewards, group advantages, token masks, and experiment artifacts. A job can finish normally while one of these boundaries silently changes the treatment or erases the intended signal. We call this a *silent reward-contract failure* and present ARCA, a CPU-only preflight auditor for typed trace equivalence, action–result pairing, reward flow and informativeness, grouped-advantage support, mask identity, status, and provenance. The study separates discovery from frozen evaluation. Six natural cases are reproduced across five source-pinned systems; two change a registered scientific conclusion. On an unseen OpenRLHF revision, frozen rules detect all 40 registered single-fault mutations across four families, while flagging none of 40 source-derived clean controls; native preflight detects 25% of the mutations. A later five-system purposive validation finds two additional natural cases and yields 7/7 exact case predictions with 0/5 clean false positives, whose Wilson 95% interval is nevertheless $[0.0000, 0.4345]$. We therefore claim case-based external transfer, not ecosystem prevalence or reliability certification. A preregistered GPU attempt failed before model loading and is reported as infrastructure-invalid, not as evidence about learning. The contribution is an executable audit methodology, a source-pinned case corpus, and fail-closed reporting practices for establishing whether an agentic RL experiment implemented its intended reward contract. Our results motivate contract preflight as a validity check before learning curves are interpreted, while leaving downstream model effects for a separately valid experiment.

1 Introduction

Multi-turn agent training is no longer a single function from prompt to scalar reward. A rollout may cross a serializer, tool protocol, environment adapter, reward worker, grouping rule, token selector, and asynchronous storage layer before it becomes a gradient update. Recent work improves reward design (Zhang et al., 2026; Modecrua et al., 2026), credit assignment (Zhang, 2026), and training stability (Wang et al., 2025). These advances generally assume that the value called "reward," the trajectory being scored, and the tokens being optimized still represent the intended experiment.

That assumption can fail without a crash. A JSON object can become a quoted string, a missing reward can become the valid scalar zero, a producer can write one reward field while a consumer reads another, or two nominal mask arms can select the same response tokens. Standard checks—successful exit, finite loss, schema validity, or non-empty output—need not expose these errors. The run is operationally successful but scientifically uninterpretable.

We study this earlier validity question:

> Did the executed pipeline preserve the intended trace, reward, comparison, treatment, and evidence contracts strongly enough for the resulting learning claim to be meaningful?

We make four contributions. First, we define a contract chain and eight executable failure rules spanning six families. Second, we provide ARCA, a small framework-independent auditor with explicit positive controls. Third, we evaluate frozen rules on source-pinned natural cases, registered mutations, an unseen framework, and a later purposive five-system sample. Fourth, we release a fail-closed artifact that preserves failed and unopened runs instead of selectively repairing or excluding them.

Our claim is deliberately narrower than a prevalence study or a new RL algorithm. The evidence shows that silent contract failures occur in multiple public systems and that the registered checks transfer to specific unseen boundaries. It does not estimate how often such failures occur in the field, show that audited projects are broadly unreliable, or demonstrate improved model quality.

2 Related Work and the Remaining Gap

Reward quality and reward exploitation. Checklist rewards and iterative calibration make feedback more informative for multi-turn tool use (Zhang et al., 2026; Modecrua et al., 2026). Reward-hacking benchmarks instead test whether agents exploit evaluators or task shortcuts (Thaman, 2026). Both ask whether the objective is useful or gameable. Our failure unit is different: an honest trajectory and intended evaluator can be corrupted by a software boundary before optimization.

Credit assignment and agentic RL systems. Credit-assignment work distributes outcome information across tokens, steps, turns, or agents (Zhang, 2026). RAGEN and Agent-R1 expose trajectory and optimization abstractions (Wang et al., 2025; Cheng et al., 2025). Agent Lightning and OpenRLHF separate execution, reward, and training interfaces (Luo et al., 2025; OpenRLHF Contributors, 2026). veRL and AReaL provide multi-turn or asynchronous training boundaries (veRL Contributors, 2026; AReaL Contributors, 2026); slime and rLLM provide further reward and evaluator interfaces (slime Contributors, 2026; rLLM Contributors, 2026). Modularity creates explicit boundaries but does not itself test cross-boundary semantic preservation.

Pipeline evaluation. Agent^2 RL-Bench evaluates whether coding agents can build and close an RL loop under a budget (Chen et al., 2026). ARCA instead evaluates a pipeline supplied by a researcher: its unit is a typed contract and a positive control, not the engineering performance of an autonomous coding agent.

The closest methodological comparators are source-framework preflights and Agent^2 RL-Bench. Native preflights validate one framework's accepted inputs, while Agent^2 RL-Bench scores whether an engineering agent closes an RL loop. Neither evaluates whether a supplied experiment preserves typed trace meaning, reward flow, advantage support, and treatment identity across framework boundaries. Conversely, ARCA does not evaluate autonomous engineering ability or propose a training algorithm. This distinction is the claimed novelty delta.

Our refreshed search over arXiv, OpenReview, official documentation, and official repositories applied five substitute criteria: typed trace checks, reward and advantage checks, treatment-mask identity, cross-framework execution, and held-out framework evaluation. No screened work satisfied all five. The remaining gap is therefore a frozen, executable preflight for semantic experiment contracts, evaluated on source-pinned cases and an unseen framework. This is a bounded search conclusion, not proof of universal novelty.

3 Reward Contracts

Let an intended experiment be the contract chain in Equation 1:

$$x \xrightarrow{S} \tau \xrightarrow{E} r \xrightarrow{G} A \xrightarrow{M} z \xrightarrow{L} \mathcal{A}, \tag{1}$$

where S serializes an interaction trace τ, E evaluates it, G forms a comparison or advantage A, M selects trainable tokens z, and L writes the evidence artifact $\mathcal{A}$. A contract C_j specifies an invariant at one boundary and an executable positive control that must distinguish at least two intended inputs.

We define a *silent reward-contract failure* as a violation of some C_j for which the enclosing process can still terminate successfully and emit superficially valid outputs. This excludes loud exceptions that remain visible

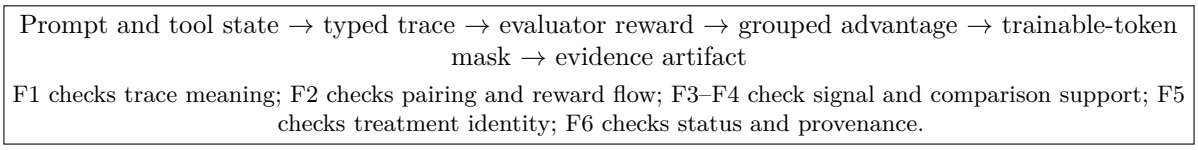

Figure 1: The audited contract chain from executed interaction to scientific artifact. A process-level success signal can coexist with a semantic failure at any boundary.

Table 1: Registered contract families. F2 and F6 each contain two executable rules, giving eight rule codes across six families.

Family	Rule	Boundary	Positive-control invariant
F1	TYPE	trace serialization	Equivalent tool arguments canonicalize to one JSON object.
F2	PAIRING	trace assembly	Tool calls and results have equal identifier multisets.
F2	REWARD-FLOW	reward adapter	Intended keys are both produced and consumed.
F3	INFORMATIVE	evaluator output	Registered contrasting controls yield finite, distinct rewards.
F4	ADVANTAGE	group construction	Every comparison group has at least two items and some reward support.
F5	TREATMENT	token masking	Distinct, non-aliased arms do not select identical token masks.
F6	STATUS	failure coercion	A numeric fallback retains a structured failure or missingness status.
F6	PROVENANCE	artifact boundary	Observed source and protocol identifiers equal required identifiers.

in structured status. It also excludes explicit, documented fallback behavior when its occurrence remains observable to downstream analysis.

Table 1 lists the registered families. In the stable codes, we use typed-argument equivalence (TYPE), reward transport (REWARD), flow continuity (FLOW), and not scientifically evaluated (NOT). The implementation returns these codes rather than framework-specific error messages.

4 ARCA Auditor

ARCA consumes one schema-valid case containing a framework, workload, seed, evidence class, expected rule codes, and a payload. Each rule is a deterministic predicate. For example, TYPE parses string arguments and canonicalizes JSON objects; PAIRING compares identifier multisets; REWARD-FLOW intersects intended, produced, and consumed keys; INFORMATIVE requires finite contrasting rewards; and TREATMENT compares masks after declared aliases are removed. STATUS fires only when a failure is represented as numeric reward without structured status. PROVENANCE compares required and observed identifiers.

The design is intentionally small. It neither imports an RL framework nor attempts to infer a user's objective from logs. Framework adapters extract facts from source-pinned boundaries into the common schema. This separates the auditable rule from the code needed to observe a particular system.

Fail-closed execution. Every generator records a protocol identifier, deterministic case identity, source hashes, execution state, and finite measurements. Duplicate steps, unaligned arms, source drift, or frozen-evaluator drift invalidate an artifact. Failures and unopened runs remain outcomes. A controller may stop a bounded experiment, but it may not silently repair and splice results into the consumed protocol.

5 Study Design

The study uses four evidence classes and never pools their denominators. These classes instantiate the boundaries in Figure 1:

1. **Natural cases**: behavior reproduced from a pinned production or documented execution boundary without inserting a synthetic fault.
2. **Source-derived clean controls**: valid states constructed from the same inspected boundary.
3. **Registered mutations**: one predeclared fault inserted into an otherwise clean case.
4. **Post-freeze replications**: cases found after the primary rule and held-out evaluation were frozen.

5.1 Development and freeze

The development set contains 163 cases. Three are natural cases from AReno and veRL; the remaining 160 are balanced clean controls and single-fault mutations over eight rule codes and ten fixture seeds. Before inspecting the held-out framework, we froze the auditor, evaluator, and development cases by the Secure Hash Algorithm (SHA)-256. Development macro recall was 1.00. The clean false-positive rate (FPR) was 0/80 with Wilson 95% interval $[0.0000, 0.0458]$, and all three natural cases were detected.

5.2 One-shot held-out evaluation

The held-out system is OpenRLHF at commit `bc71bb19464aca306b33080b2d2bb45d154e2f49`. Its adapter was written only after the freeze. Inspection produced 40 source-derived clean controls and 40 registered single-fault mutations across reward flow, group advantage, treatment identity, and failure status. No natural OpenRLHF failure was found; therefore this split measures mutation transfer, not natural-case prevalence.

We compare ARCA with exit-only, finite-only, schema-only, and the native preflight observable at the inspected OpenRLHF boundary. The primary metrics are macro recall across mutation families and clean false-positive rate. We report Wilson intervals and a preregistered hierarchical bootstrap over the framework–workload–family cells; repeated fixture seeds are measurements, not independent framework samples.

5.3 External natural validation

After the held-out evaluation, five previously unseen systems were selected by frozen inclusion criteria before their relevant files were opened: slime, Agent-R1, RAGEN, rLLM, and Agent Lightning. All remained in the denominator after inspection. Qualifying natural behavior was dynamically reproduced by compiling and executing only the pinned function bodies. A framework-level clean control was retained for every candidate, including negative inspections. This sample is purposive and small.

6 Natural Case Studies

Table 2 reports all six natural cases. A conclusion flip means that the observed behavior changes a preregistered treatment distinction or a gate, not necessarily downstream model quality.

The rLLM behavior occurs in two production coercion functions but is counted once because both instantiate the same evaluator contract. The slime case is an official evaluation-replay boundary, not a core training-path finding. Agent-R1, RAGEN, and Agent Lightning are bounded negative inspections: their audited fallbacks retained structured errors, warnings, or reward-presence evidence and were not relabelled as findings.

7 Results

7.1 Frozen held-out transfer

Table 3 shows the one-shot result. ARCA detects 10/10 mutations in each of four held-out families, for macro recall 1.00. It flags 0/40 source-derived clean controls; the Wilson 95% interval for clean FPR is $[0.0000, 0.0876]$. Exit-only, finite-only, and schema-only checks detect none of the registered semantic faults. The native preflight detects one family, for macro recall 0.25. The hierarchical bootstrap interval for ARCA macro recall is $[1.00, 1.00]$ under the frozen registered cells; it does not cover unregistered fault families.

Table 2: Natural cases, kept separate by boundary and evidence phase.

Phase	System	Rule	Severity	Reproduced behavior	Flip
Dev	AReno	F1 TYPE	high	Tool argument alternatives are not equivalent JSON objects.	yes
Dev	veRL	F2 REWARD-FLOW	high	Intended tool reward is produced under a field the consumer does not use.	yes
Dev	veRL	F6 STATUS	medium	Timeout is converted to numeric zero without structured status.	no
Post-freeze	AReaL	F6 STATUS	medium	Missing reward becomes zero in the assembled tensor without missingness status.	no
P5	slime	F6 STATUS	medium	Missing evaluation-replay reward becomes 0.0 with no reward-present field.	no
P5	rLLM	F3 INFORMATIVE	high	Dicts lacking `reward` map to 0.0 even when `is_correct` differs.	yes

Table 3: Held-out OpenRLHF mutation transfer. Each method has clean FPR 0/40; the shared point estimate must be read with its $[0.0000, 0.0876]$ Wilson interval.

Auditor	Mutation macro recall
Exit-only	0.00
Finite-only	0.00
Schema-only	0.00
Native preflight	0.25
ARCA	1.00

7.2 External validation

Table 4 retains every selected framework. The five-system external sample contains two natural cases and five clean controls. Frozen rules detect both natural cases, including the one registered high-severity case, and produce the exact expected rule set on all 7 cases. The clean FPR is 0/5. Its Wilson 95% interval is $[0.0000, 0.4345]$, which is too wide to support a claim that ecosystem FPR is low. The external result supports case-based transfer to two previously unseen boundaries and documents three negative inspections; it is not a reliability certificate.

8 Invalid Dynamic Attempt

A separately frozen GPU protocol was intended to test whether strict versus canonical reward handling changes downstream training. The first of six commands failed after 12.8175 seconds because the reward module could not import the project package. The failure occurred before model loading, rollout generation, reward evaluation, or any optimizer update. Scientific trajectories and training updates were both zero; the remaining five commands were unopened. Observed single-GPU occupancy was 0.00356 hours.

The protocol outcome and scientific status are, respectively,

```
INVALID_P4_INFRASTRUCTURE_REWARD_IMPORT
NOT_EVALUATED
```

No P4 arm difference appears in our analysis. This example also illustrates why fail-closed artifacts matter: repairing and selectively rerunning the consumed protocol would erase the actual execution history.

Table 4: External validation retains all selected systems.

System	Audited boundary	Outcome
slime	forge evaluation replay	natural F6 case
Agent-R1	WebShop environment step	bounded negative
RAGEN	Lean timeout/server error	bounded negative
rLLM	Python evaluator coercion	natural F3 case
Agent Lightning	veRL daemon reward fill	bounded negative

9 Threats to Validity

Selection and prevalence. Systems were purposively selected for relevant agentic-RL boundaries and public inspectability. Six cases across five systems establish existence and heterogeneity, not prevalence. The external 0/5 clean result has a 0.4345 upper confidence bound and cannot certify low FPR.

Mutation construct validity. The held-out score uses registered single-fault mutations. These isolate rule behavior but may not represent the joint faults, concurrency effects, or version-specific interactions seen in production. Perfect registered recall is not universal coverage.

Source-to-runtime validity. Natural cases were tied to pinned source and, where feasible, exact function bodies on CPU. We did not run full distributed training for these inspections. Compiler extraction can miss effects introduced by surrounding services. We therefore describe exact audited boundaries and avoid project-wide claims.

Rule dependence. Adapters encode which facts are observable. A malformed adapter can hide or create a violation. Source hashes, common schemas, positive controls, and held-out freeze reduce but do not eliminate this risk.

No downstream model claim. P4 generated no scientific trajectory. We do not know the size or direction of any model-quality effect. The present contribution is preflight validity, not improved optimization.

10 Responsible Reporting and Broader Impact

Publishing source-pinned software cases can help researchers avoid invalid RL runs and wasted compute. It can also be misread as a security disclosure or a ranking of framework quality. We mitigate this risk by reporting bounded functions and commits, retaining negative inspections, separating explicit fallbacks from silent failures, and avoiding vulnerability-rate language. No private system, user data, model output, or credential is included. Public maintainer contact and disclosure are outside this artifact and require a separate decision.

The auditor can itself create false confidence if users treat passing eight rules as complete correctness. The intended use is as an extensible preflight with registered positive controls, followed by task-specific validation. It is not a general reward-hacking defense, formal verification system, or safety certificate.

11 Conclusion

Agentic RL experiments can preserve operational success while losing the scientific meaning of a trace, reward, comparison, or treatment. ARCA turns a narrow set of these assumptions into eight executable rules. Frozen rules detected all 40 held-out mutations with 0/40 clean flags, and later matched two natural cases plus five controls without rule changes. These results are bounded by mutation dependence, purposive system selection, a wide external-control interval, and the absence of a valid downstream training study. Future work should test independently contributed contracts and, under a separately frozen protocol, measure whether fail-closed preflight prevents invalid runs at acceptable overhead. Before interpreting a learning

curve, researchers should verify that contrasting traces reach contrasting rewards, advantages, masks, and artifacts under pinned provenance.

References

AReaL Contributors. AReaL: A large-scale asynchronous reinforcement learning system. Source repository and official documentation, commit 62f955c5e0388aebc6fd58c5ad3f8fcf9d7384b8, 2026. URL https://github.com/inclusionAI/AReaL.

Wanyi Chen, Xiao Yang, Xu Yang, Tianming Sha, Qizheng Li, Zhuo Wang, Bowen Xian, Fang Kong, Weiqing Liu, and Jiang Bian. Agent^2 RL-bench: Can LLM agents engineer agentic RL post-training? *arXiv preprint arXiv:2604.10547*, 2026. URL https://arxiv.org/abs/2604.10547.

Mingyue Cheng, Shuo Yu, Daoyu Wang, Qingchuan Li, Xiaoyu Tao, Jie Ouyang, Yucong Luo, Yitong Zhou, Qi Liu, and Enhong Chen. Agent-R1: A unified and modular framework for agentic reinforcement learning. *arXiv preprint arXiv:2511.14460*, 2025. URL https://arxiv.org/abs/2511.14460.

Xufang Luo, Yuge Zhang, Zhiyuan He, Zilong Wang, Siyun Zhao, Dongsheng Li, Luna K. Qiu, and Yuqing Yang. Agent lightning: Train any AI agents with reinforcement learning. *arXiv preprint arXiv:2508.03680*, 2025. URL https://arxiv.org/abs/2508.03680.

Wachiravit Modecrua, Krittanon Kaewtawee, Krittin Pachtrachai, and Touchapon Kraisingkorn. Multi-turn reinforcement learning for tool-calling agents with iterative reward calibration. *arXiv preprint arXiv:2604.02869*, 2026. URL https://arxiv.org/abs/2604.02869.

OpenRLHF Contributors. OpenRLHF: An easy-to-use, scalable and high-performance RLHF framework. Source repository, commit bc71bb19464aca306b33080b2d2bb45d154e2f49, 2026. URL https://github.com/OpenRLHF/OpenRLHF.

rLLM Contributors. rLLM: Reinforcement learning for language agents. Source repository, commit 4d81632871fe4c38dadced26529a1f81833e2e99, 2026. URL https://github.com/rllm-org/rllm.

slime Contributors. slime: An LLM post-training framework for RL scaling. Source repository, commit 526f87880b197bdcc5a28d621258eeeb557866df, 2026. URL https://github.com/THUDM/slime.

Kunvar Thaman. Reward hacking benchmark: Measuring exploits in LLM agents with tool use. *arXiv preprint arXiv:2605.02964*, 2026. URL https://arxiv.org/abs/2605.02964.

veRL Contributors. veRL: Volcano engine reinforcement learning for LLMs. Source repository and official documentation, commit e9618406de5bad40041d7612554e465ec2003ec1, 2026. URL https://github.com/volcengine/verl.

Zihan Wang, Kangrui Wang, Qineng Wang, Pingyue Zhang, Linjie Li, Zhengyuan Yang, Xing Jin, Kefan Yu, Minh Nhat Nguyen, Licheng Liu, Eli Gottlieb, Yiping Lu, Kyunghyun Cho, Jiajun Wu, Li Fei-Fei, Lijuan Wang, Yejin Choi, and Manling Li. RAGEN: Understanding self-evolution in LLM agents via multi-turn reinforcement learning. *arXiv preprint arXiv:2504.20073*, 2025. URL https://arxiv.org/abs/2504.20073.

Chenchen Zhang. From reasoning to agentic: Credit assignment in reinforcement learning for large language models. *arXiv preprint arXiv:2604.09459*, 2026. URL https://arxiv.org/abs/2604.09459.

Zhen Zhang, Kaiqiang Song, Xun Wang, Yebowen Hu, Weixiang Yan, Chenyang Zhao, Henry Peng Zou, Haoyun Deng, Sathish Reddy Indurthi, Shujian Liu, Simin Ma, Xiaoyang Wang, Xin Eric Wang, and Song Wang. CM2: Reinforcement learning with checklist rewards for multi-turn and multi-step agentic tool use. *arXiv preprint arXiv:2602.12268*, 2026. URL https://arxiv.org/abs/2602.12268.

A Reproducibility Checklist

The anonymous artifact contains the frozen evaluator, case generators, source manifests, JSON and CSV results, exact CPU commands, and a deterministic archive builder. Every distributed file is allowlisted and hashed. A verifier rejects unexpected files, hash drift, non-parsing JSON, headerless CSV, local absolute paths, identity strings, Secure Shell (SSH) endpoints, credential-like assignments, and Git metadata. The invalid P4 evidence archive is deliberately excluded; only its redacted gate summary is distributed.

B Interpretation Rules

For a mutation family with x detections out of n registered cases we report recall x/n. For k clean flags among m controls we report k/m and the two-sided Wilson interval. Framework–workload–family is the inferential cell; fixture seeds are repeated measurements. Missing, failed, or unopened runs remain explicit terminal outcomes. Natural and synthetic evidence is never aggregated into a vulnerability rate.

C Source Pins

The central frozen revisions are:

veRL `e9618406de5bad40041d7612554e465ec2003ec1`

OpenRLHF `bc71bb19464aca306b33080b2d2bb45d154e2f49`

AReaL `62f955c5e0388aebc6fd58c5ad3f8fcf9d7384b8`

The external manifest records the pinned revisions and SHA-256 hashes for slime, Agent-R1, RAGEN, rLLM, and Agent Lightning. These identifiers define the scope of every technical claim; later upstream behavior may differ.